코딩 스타일 가이드

태그	가이드와 프로세스

✅ Code Style Guide (2026)
- 📌 요약
- ✅ 1. 명명 (Naming)
  - ✅ 기본 규칙
  - ✅ 접두어/동사 규칙
  - ✅ 이벤트 네이밍 (React)
  - ✅ boolean 네이밍
  - ✅ 케이스/파일명
- ✅ 2. 선언 (Declarations)
  - ✅ var 금지
  - ✅ 체이닝 할당 금지
  - ✅ 선언 위치/순서
  - ✅ ++/-- 규칙
- ✅ 3. 객체 (Objects)
  - ✅ 리터럴/단축형/계산된 키
  - ✅ hasOwn

✅ Code Style Guide (2026)

> React 19 / Next(App Router) / TypeScript 기준
>
> 목표: **리뷰 기준 통일 · 유지보수 비용 감소 · 버그 예방**

📌 요약

- `const` 우선, `let`은 재할당 필요할 때만
- 이벤트 네이밍: **props는 `onX` / 내부 핸들러는 `handleX`**
- boolean 네이밍: `is/has/can/should`
- `any` 금지 → `unknown` + 타입가드
- Server/Client 경계 엄수(클라이언트 혹은 Client Component에서만)
- `===` 사용, 값 비교는 의도를 명확히
- default export 최소화, named export 권장
- 포맷은 Prettier/ESLint가 결정(논쟁 금지)

✅ 1. 명명 (Naming)

✅ 기본 규칙

- 숫자/한국어 발음 기반 표기 지양
- 의미가 드러나는 이름 사용

사용
✅ spread/rest 선호
✅ 4. 배열 (Arrays)
✅ 배열 생성/추가/복사
✅ 변환은 Array.from / spread
✅ 콜백 return 명확히
✅ 5. 구조 분해 (Destructuring)
✅ 6. 문자열 (Strings)
✅ 7. 함수 (Functions)
✅ 기본 규칙
✅ export 전략
✅ 8. 화살표 함수 (Arrow Functions)
✅ 9. 속성 접근 (Properties)
✅ 10. 비교 연산자 및 동등성 (Comparisons)
✅ 불필요한 삼항 지양
✅ ?? 사용
✅ 11. 제어문 (Control Flow)
✅ 12. 주석 (Comments)

```
// bad
function q() {}

// good
function queryProjects() {}
```

✅ 접두어/동사 규칙

- `get`: 조회(가능하면 side-effect 없음)
- `set`: 대입/상태 변경
- `is/has/can/should`: boolean 반환
- `create/update/remove`: 생성/수정/삭제
- `fetch/load`: I/O 포함(네트워크/DB/스토리지)
- `toggle`: 토글

```
const isValid = isEmail(email);
const user = awaitfetchUser(userId);
await updateProject(projectId, payload);
```

✅ 이벤트 네이밍(React)

- `onX`: props로 내려오는 콜백
- `handleX`: DOM 이벤트를 받아 `onX` 호출하는 내부 핸들러

```
// Parent
function Counter() {
  const [count, setCount] =useState(0);

  const onCountChange = (offset:number) ⇒ {
    setCount((c) ⇒ c + offset);
 };

  return <ButtonPlus onCountChange={onCountChange} /
>;
}
```

✅ 13. 공백/포맷팅 (Whitespace)
✅ 14. 타입 변환 (Type Conversion)
🔥 Next/App Router 필수 규칙
✅ Server/Client 경계
🧷 TypeScript 안전 규칙
✅ any 금지 → unknown + 타입가드
✅ Import 규칙 (프로젝트 컨벤션)
✅ 팀 체크리스트(리뷰용)

```
// Child
function ButtonPlus({ onCountChange }: { onCountChange:
(offset: number) ⇒void }) {
    const handleClick = () ⇒ onCountChange(1);
    return<button onClick={handleClick}>+</button>;
}
```

✅ boolean 네이밍

- boolean은 상태/판단이 읽히게

```
// bad
dragon.isAge();

// good
dragon.hasAge;
dragon.isAdult;
isAgeValid(age);
```

✅ 케이스/파일명

- 변수/함수: `camelCase`
- 타입/컴포넌트/클래스: `PascalCase`
- 상수(진짜 상수만): `SCREAMING_SNAKE_CASE`

```
export const API_BASE_URL ="https://api.example.com";

type UserProfile = { id:string };
function LoginForm() {}
```

✅ 2. 선언 (Declarations)

✅ var 금지

```
// bad
var a = 1;

// good
const a = 1;
let count = 0;
count += 1;
```

✅ 체이닝 할당 금지

```
// bad
let a = (b = c =1);

// good
const c = 1;
const b = c;
const a = c;
```

✅ 선언 위치/순서

- `const` → `let`
- 사용 근처에 선언

```
// good
function checkName(hasName:string) {
    if (hasName ==="test") return false;

    const name = getName();
    if (name ==="test") return false;

    return name;
}
```

✅ `++/--` 규칙

- 단독 라인 허용

- 표현식 안에서 부수효과 발생은 지양

```
// ok
i++;

// bad
array[i++];

// good
const value = array[i];
i += 1;
```

✅ 3. 객체 (Objects)

✅ 리터럴/단축형/계산된 키

```
// good
const name = "Luke";

const obj = {
  name,
    getName() {
        return name;
  },
  ["isEnabled"]: true,
};
```

✅ hasOwn 사용

```
const example:Record<string,unknown> = {};
Object.hasOwn(example,"prop");// false
```

✅ spread/rest 선호

```
const original = { a:1,b:2 };
const copy = { ...original,c:3 };
```

```
const { a, ...rest } = copy;
```

✅ 4. 배열 (Arrays)

✅ 배열 생성/추가/복사

```
const items:number[] = [];
items.push(1);

const copy = [...items];
```

✅ 변환은 Array.from / spread

```
const arrLike = { 0:"a",1:"b",length:2 };
const arr = Array.from(arrLike);
```

✅ 콜백 return 명확히

```
// good
const filtered = inbox.filter((msg) => {
    if (msg.subject === "Mockingbird") {
        return msg.author === "Harper Lee";
  }
    return false;
});
```

✅ 5. 구조 분해 (Destructuring)

```
// good
functi ongetFullName({ firstName, lastName }: { firstName: s
tring; lastName:string }) {
    return `${firstName} ${lastName}`;
}
```

```
const arr = [1,2,3];
const [first, second] = arr;
```

✅ 6. 문자열 (Strings)

- `eval()` 금지
- 불필요한 escape 금지

```
// bad
eval("alert(1)");

// good
JSON.parse(text);
```

✅ 7. 함수 (Functions)

✅ 기본 규칙

- 매개변수 직접 변경 금지
- `Function` 생성자 금지
- `arguments` 금지 → rest 사용

```
// good
function sum(...args:number[]) {
    return args.reduce((a, b) => a + b, 0);
}
```

✅ export 전략

- named export 권장
- default export 최소화

```
export function parseApiError() {}
```

✅ 8. 화살표 함수 (Arrow Functions)

```
const inc = (x:number) ⇒ x +1;

const toMap = (x:number,index:number) ⇒ ({ [index]: x });

const build = (x:number) ⇒ {
    const y = x +1;
    return y * x;
};
```

✅ 9. 속성 접근 (Properties)

- 일반 접근: dot
- 변수 키: bracket

```
const luke = { jedi:true };
const isJedi = luke.jedi;

function getProp<Textendsobject, Kextends keyof T>(obj: T,
key: K) {
    return obj[key];
}
```

✅ 10. 비교 연산자 및 동등성 (Comparisons)

- `===`, `!==` 사용
- boolean은 축약형
- 비교는 의도 명확히

```
if (isValid) {}

if (name.length > 0) {}

if (items.length > 0) {}
```

✅ 불필요한 삼항 지양

```
// bad
const bar = c ? true :false;

// good
const bar = Boolean(c);
```

✅ `??` 사용

```
const age = user.age ?? 18;
```

✅ 11. 제어문 (Control Flow)

- early return 우선
- 단축평가로 제어문 대체 지양

```
// bad
!isRunning && startRunning();

// good
if (!isRunning) {
    startRunning();
}
```

✅ 12. 주석 (Comments)

- 주석은 “왜(why)” 중심
- 여러 줄은 `/** */`

```
/**
 * 외부 API rate limit이 있어 캐시를 우선 사용한다.
 */
export function getBillingHistory() {}
```

✅ 13. 공백/포맷팅 (Whitespace)

- 블록 뒤 빈 줄 허용(가독성)
- 블록 내부 불필요 빈 줄 금지
- 줄 끝 공백 금지(Prettier로 강제)

```
if (foo) {
    return bar;
}

return baz;
```

✅ 14. 타입 변환 (Type Conversion)

```
const s = String(value);
const n = Number(input);
const i = parseInt(input,10);
const ok = Boolean(value);// 또는 !!value (팀 통일)
```

🔥 Next/App Router 필수 규칙

✅ Server/Client 경계

- Zustand / React Query / react-hook-form 훅은 **Client Component에서만**
- Server Component에서 client store 훅 호출 금지

```
// bad (Server Component)
export default function Page() {
    const s =useUiStore(); // ❌
    return null;
}

// good
import { Sidebar } from "@/widgets/sidebar";
```

```
export default function Page() {
  return <Sidebar />; // ✅ 내부에서 SidebarClient가 훅 사용
}
```

📌 TypeScript 안전 규칙

✅ any 금지 → unknown + 타입가드

```
function isRecord(v:unknown): v isRecord<string,unknown>
{
  return typeof v ==="object" && v !== null;
}
```

✅ Import 규칙(프로젝트 컨벤션)

- public API(index.ts) 경유 import 우선
- 딥 임포트 지양

```
// good
import { Sidebar } from "@/widgets/sidebar";

// bad
import { SidebarClient } from "@/widgets/sidebar/ui/Sidebar
Client";
```

✅ 팀 체크리스트(리뷰용)

- ☐ 이벤트 네이밍 `onX / handleX` 지켰나?
- ☐ `any` 쓰지 않았나? (`unknown` + 가드)
- ☐ Server/Client 훅 호출 위치 맞나?
- ☐ 비교/조건문이 의도 명확한가?

- [ ] export는 named 위주인가?
- [ ] 딥임포트 없이 public API로 가져왔나?